

บทที่ 3

องค์ประกอบของระบบกริด(Grid Component)

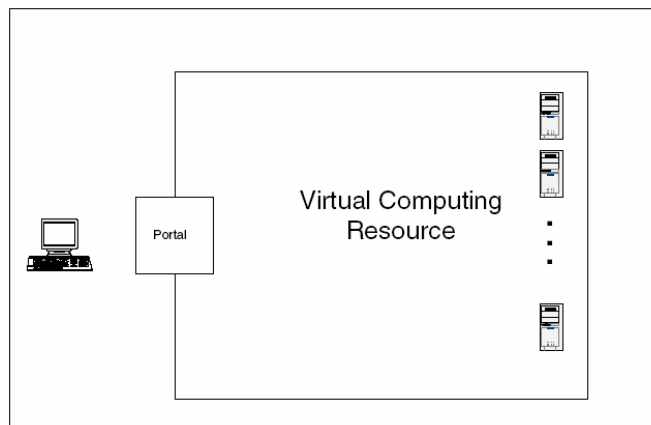
และ GLOBUS toolkit

3.1. องค์ประกอบของระบบกริด(Grid Component)

3.1.1. portal – user interface

เปรียบเสมือนเป็นกำแพงชั้นแรกในการเข้าใช้ระบบกริดผู้ใช้ จะมองไม่เห็นการทำงานภายในระบบเป็นส่วนของการติดต่อระหว่างผู้ใช้ระบบกริดและตัวระบบ เพื่อให้ผู้ใช้สามารถเรียกใช้แอปพลิเคชันขึ้นมาทำงานได้ ในมุมมองของผู้ใช้ระบบกริดจะมองระบบเป็นเหมือนแหล่งทรัพยากรเสมือน ซึ่งใน globus toolkit จะไม่มีส่วนของพอร์ทัล (Portal) นี้จึงจะเห็นได้จากรูปที่ 3-1 หากต้องการเรียกใช้จะต้องใช้เครื่องมืออื่นๆเข้ามาช่วย อาทิเช่น Websphere

พอร์ทัลอาจจะนำเสนอในรูปแบบของ เว็บเพจ(web page) ซึ่งเป็นรูปแบบที่ง่ายในการจัดการหรือติดต่อกับแอปพลิเคชันของระบบกริด ซึ่งในพอร์ทัลนี้จะมีส่วนของการพิสูจน์ตน ,การเสนองาน , การดูแลงานที่ทำอยู่ และการแสดงผลของงานนั้นๆ ส่วนที่มีความสำคัญมากที่สุดของพอร์ทัล คือทำอย่างไรให้ผู้ใช้งานได้รับความสะดวกในการใช้งาน



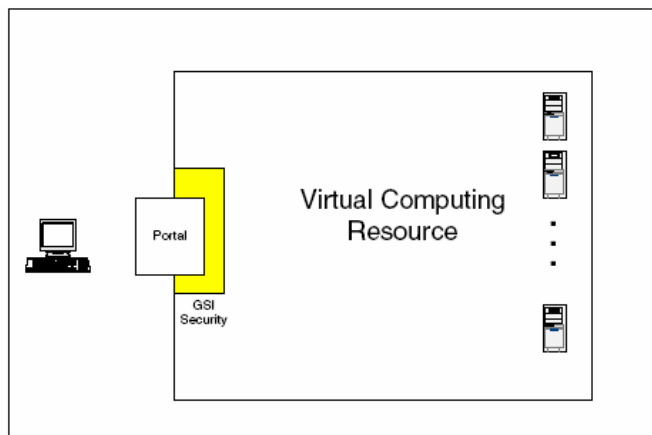
รูปที่ 3-1 ตำแหน่งของพอร์ทัลในระบบกริด

3.1.2. Security (ระบบรักษาความปลอดภัย)

องค์ประกอบที่สำคัญอย่างหนึ่งคือเรื่องของความปลอดภัย จะใช้บริการหลัก 3 บริการด้วยกัน คือ การพิสูจน์ตัวตน , การพิสูจน์สิทธิ์ , การเข้ารหัสลับ ซึ่งใน globus จะเรียกองค์ประกอบนี้ว่า GSI ดังแสดงให้

เห็นในส่วนที่แรเงาในรูปที่ 3-2 ซึ่งใน GSI จะใช้ OpenSSL ในการสร้าง ตลอดจนมีกลไกของซิงเกิ้ลไชน์ออนไลน์

ในระบบกริด เมื่อต้องการเรียกใช้ทรัพยากรด้วยการทำงานกับระบบที่อยู่ระยะไกล จะต้องมั่นใจได้ว่าการทำงานแบบนี้จะไม่มีใครได้ล่วงรู้ข้อมูลที่กำลังทำการส่งอยู่ และหากเครื่องที่ปฏิบัติงานอยู่เป็นผู้ให้บริการด้านทรัพยากร จะต้องมั่นใจว่าถ้ามีผู้ใช้ระบบเข้ามาทำงานที่เครื่องนี้ จะไม่ทำให้เกิดการรบกวนเจ้าของเครื่อง หรือแม้กระทั่งสามารถเข้าถึงข้อมูลของระบบได้ ซึ่งระบบรักษาความปลอดภัย ภายใน globus จะประกอบไปด้วยการพิสูจน์ตน การพิสูจน์สิทธิ์ และการติดต่อกันระหว่างทรัพยากรอย่างปลอดภัย



รูปที่ 3-2 ตำแหน่งของ GSI ในระบบกริด

- **การพิสูจน์ตน** สามารถเรียกใช้ซิงเกิ้ลไชน์ออนไลน์โดยการใช้คำสั่ง `grid-proxy-init` เพื่อสร้างพร็อกซี่ชั่วคราวโดยการสร้างพร็อกซี่ นี้จะใช้คีย์ส่วนตัวของผู้ที่ทำการสร้างพร็อกซี่ เมื่อสร้างเสร็จแล้ว เป็นหน้าที่ของเครื่องเซิร์ฟเวอร์ที่จะตัดสินใจในการพิสูจน์สิทธิ์ของผู้ใช้ระบบคนนั้น พร็อกซี่ จะถูกสร้างก่อนที่ผู้ใช้ระบบจะสามารถทำงานหรือส่งข้อมูลได้
- **การพิสูจน์สิทธิ์** เมื่อทำการพิสูจน์ตนแล้ว จะต้องทำการพิสูจน์สิทธิ์เป็นลำดับต่อไป เมื่อมีการร้องขอการทำงานใดๆผ่าน พร็อกซี่ ซึ่งใน พร็อกซี่ จะมีชื่อของผู้ใช้ที่เรียกว่า DN จะต้องทำการตรวจสอบชื่อ DN ให้ตรงกับ ชื่อผู้ใช้ระบบที่เป็น local
- **การติดต่อกันระหว่างกันภายในกริด** ถ้าหากว่าต้องการเข้าถึง เครื่องเซิร์ฟเวอร์หรือเครื่องไคลเอนท์ อาจจะต้องเรียกใช้การติดต่อแบบเครือข่ายระยะไกล เพื่อการเข้าใช้ระบบกริด ตัวอย่าง เช่น ใช้การ `telnet` ย่อมไม่อาจมั่นใจได้ว่าจะไม่ถูกขโมยข้อมูลออกไปจากระบบเครือข่าย ซึ่งอาจ จะใช้ `Secure Shell (SSH)` เข้ามาช่วยทำให้เกิดความปลอดภัยมากขึ้น ซึ่งใน globus มีบริการนี้อยู่แล้วแต่จะต้องเปิดบริการให้เรียบร้อยก่อน

3.1.3. Broker ,Grid Information Service , Monitoring and Discovery Service (MDS)

เมื่อทำการพิสูจน์ตัวตนเรียบร้อยแล้วผู้ใช้ระบบกริดจะทำการเรียกใช้แอปพลิเคชัน และลำดับต่อไปที่จะต้องทำคือ การระบุและแยกแยะทรัพยากรที่เหมาะสมให้กับผู้ใช้เรียกใช้แอปพลิเคชันนั้น และนี่เป็นหน้าที่รับผิดชอบของ broker ซึ่งใน globus จะไม่มี broker แต่จะมีโปรโตคอล LDAP ที่ช่วยจัดการในเรื่องของบริการ ซึ่งเรียกบริการนั้นว่า Grid Information Service(GIS) หรืออาจจะเรียกว่า Monitoring and Discovery Service(MDS) ซึ่งบริการเหล่านี้จะให้ข้อมูลเกี่ยวกับทรัพยากรที่ผู้ใช้ระบบสามารถเรียก ใช้ได้

3.1.3.1. GIS (Grid Information Service)

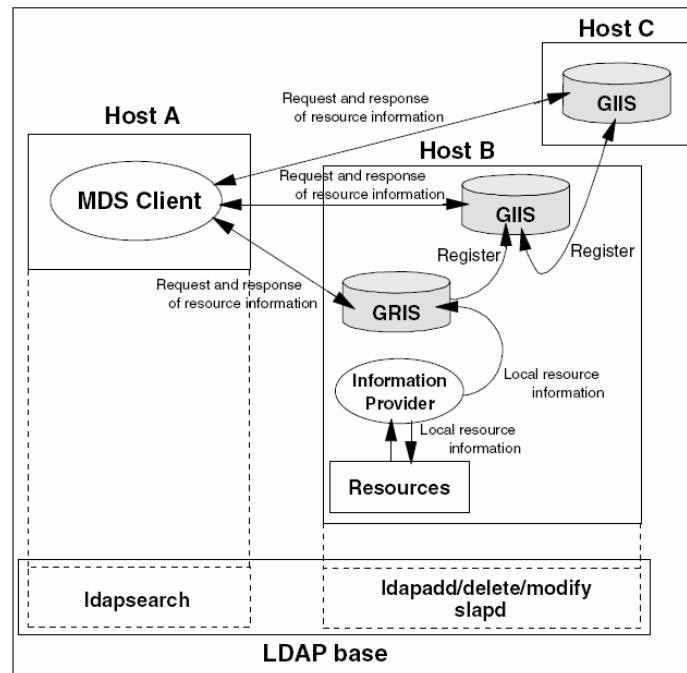
เป็นองค์ประกอบที่มีความสำคัญมากเพราะเก็บข้อมูลเกี่ยวกับทรัพยากร ขนาด และการใช้ประโยชน์ภายในกริด ซึ่งโดยปกติแล้วการใช้ประโยชน์ของหน่วยประมวลผล (ซีพียู) และทรัพยากรในกริดจะแกว่งขึ้นลงทั้งนี้ขึ้นอยู่กับโปรเซสและการใช้ข้อมูลร่วมกันซึ่งในระบบกริดจะมี grid information serviceเป็นบริการที่ให้ข้อมูลและทำการอัปเดตสถานะของทรัพยากรภายในกริดซึ่งเครื่องไคลเอนท์ broker ทำหน้าที่จัดการทรัพยากรภายในกริด จะใช้ข้อมูลจาก GIS เหล่านี้ในการตัดสินใจเลือกทรัพยากรให้เหมาะสมกับงาน

3.1.3.2. MDS (Monitoring and Discovery Service)

MDS ดังแสดงให้เห็นจากส่วนที่แรงเงาในรูปที่ 3-4 ทำหน้าที่ช่วยในการเข้าถึงข้อมูลทั้งข้อมูลสถิติ และ พลวัตน์ ของทรัพยากรในระบบกริด ซึ่งใช้มาตรฐานของโปรโตคอล LDAP (Lightweight Directory Access Protocol) องค์ประกอบของ MDS คือ

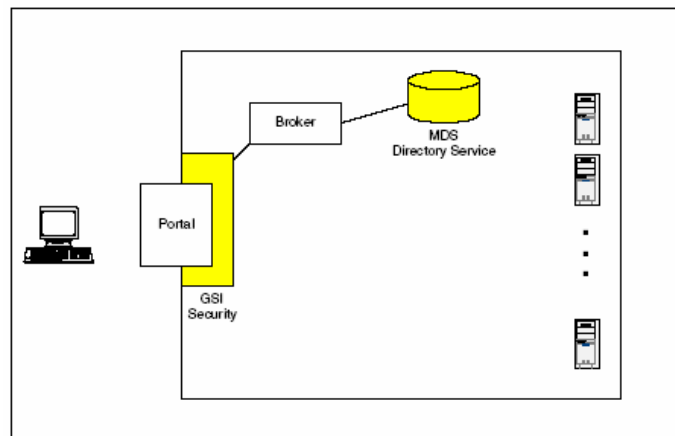
- **GRIS (Grid Resource Information Service)** เป็นแหล่งเก็บข้อมูลของทรัพยากรที่ได้รับมาจาก Information Provider GRIS สามารถลงทะเบียนข้อมูลที่มีอยู่กับ GIIS และข้อมูลของทรัพยากรนั้นจะมีเวลาอยู่ใน GRIS ในช่วงเวลาที่จำกัด คือมี TTL (Time-to-live) หาก GRIS ไม่ได้รับการร้องขอทรัพยากรเหล่านั้นเป็นเวลาเท่ากับ TTL แล้ว GRIS จะลบข้อมูลเหล่านั้นทิ้งไป และหากมีการร้องขอข้อมูลเกี่ยวกับทรัพยากรเหล่านั้นมา GRIS จะเรียกข้อมูลที่สัมพันธ์กับลักษณะข้อมูลที่ร้องขอมา และใหม่ล่าสุดจาก information provider
- **GIIS (Grid Index Information Service)** เป็นแหล่งเก็บดัชนีของข้อมูลทรัพยากร ซึ่งจะได้รับลงทะเบียนจาก GRIS หรือ GIIS ที่อื่นๆ ลักษณะของกลไกของ GIIS จะเป็นโครงสร้างระดับชั้น คล้ายกับ DNS และแต่ละ GIIS จะมีชื่อเรียกเป็นของตัวเอง จะทำให้เครื่องไคลเอนท์ที่เป็นผู้ใช้งานระบบกริดจะสามารถระบุชื่อและโหนดของ GIIS เพื่อค้นหาข้อมูลได้
- **Information Providers** ทำหน้าที่แปลงคุณสมบัติและสถานะของทรัพยากรให้อยู่ในรูปแบบที่ได้รับการกำหนดไว้

- **MDS Client** จะใช้รูปแบบเดียวกับ คำสั่ง LDAP ของเครื่องไคลเอนท์



รูปที่ 3-3 ภาพการทำงานของ MDS

จากรูปที่ 3-3 เป็นภาพรวมการทำงานของ MDS ส่งข้อมูลทรัพยากรที่ได้มาจาก Information Provider ไปยัง GRIS GRIS ทำหน้าที่ลงทะเบียนให้ข้อมูลที่ได้รับมาเป็นข้อมูลทรัพยากรของตน (local) และลงทะเบียนข้อมูลนี้ที่ GIIS และ GIIS จะทำหน้าที่ลงทะเบียนข้อมูลนี้กับ GIIS ตัวอื่นๆ และทำเช่นนั้นต่อไปเรื่อยๆ MDS client จะสามารถค้นหาข้อมูลของทรัพยากรได้โดยตรงจาก GRIS ถ้าเป็นข้อมูลของตน (local) และค้นหาข้อมูลได้จาก GIIS ถ้าเป็นข้อมูลที่อยู่แหล่งทรัพยากรอื่น (grid-wide)



รูปที่ 3-4 ตำแหน่งของ Broker และ MDS ในระบบกริด

3.1.4. Scheduler

เมื่อทรัพยากรได้รับการระบุตัวตนแล้ว อันดับต่อไปจะต้องสร้าง schedule ให้กับทรัพยากรเหล่านั้น เพื่อให้ทำงานได้ตามที่ต้องการ เมื่อมีกลุ่มของงานที่สามารถทำงานได้โดยไม่ต้องเกี่ยวเนื่องกับงานอื่น จะไม่มีความต้องการ scheduler เพื่อทำหน้าที่ในการแบ่งงาน แต่หากไม่เป็นเช่นนั้น จะต้องการ Scheduler และ inter-process communication มาช่วยในการคำนวณงานขึ้นนั้น

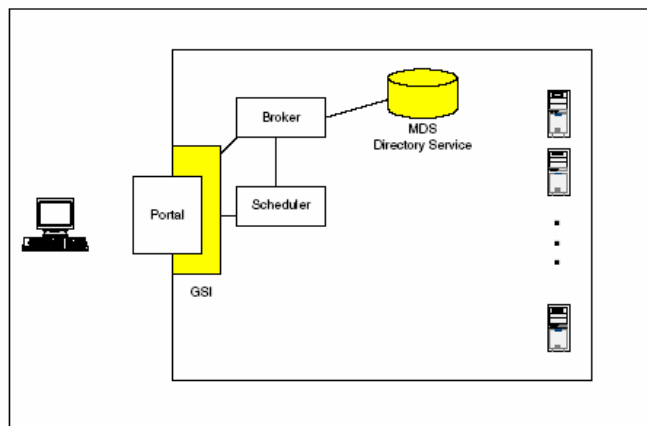
จะต้องทราบว่าในระบบกริดจะมี scheduler หลายระดับชั้น ซึ่ง scheduler ที่มีระดับชั้นสูงๆ เรียกว่า meta scheduler ใน globus toolkit จะไม่มี scheduler ให้ แต่หากต้องการให้มีจะต้องเรียกใช้ผลิตภัณฑ์อื่นๆ เช่น Condor-G

Condor-G

Condor เป็นซอฟต์แวร์ที่ประกอบไปด้วย 2 ส่วนคือ ส่วนที่ทำหน้าที่จัดการทรัพยากร และ ส่วนที่ทำหน้าที่จัดการงาน

- ส่วนที่ทำหน้าที่จัดการทรัพยากร จะช่วยทำให้เครื่องที่ทำงานอยู่สามารถทำงานได้อย่างมีประสิทธิภาพสูงสุด
- ส่วนที่ทำหน้าที่จัดการงาน เป็นส่วนที่เสนองานใหม่ให้กับระบบ หรือหยุดการทำงานแต่ยังคงเก็บค่านั้นอยู่ ตลอดจนมีข้อมูลเกี่ยวกับการจัดเรียงงาน และสถานะของงานที่ทำเสร็จเรียบร้อยแล้ว

สำหรับ Condor-G จะใช้ globus toolkit ในการเริ่มต้นทำงานที่เครื่องคอมพิวเตอร์ที่อยู่ระยะไกล แทนที่จะใช้โปรโตคอล condor ประโยชน์ของการใช้ Condor คือความสามารถในการเสนองานได้หลายงานในเวลาเดียวกัน โดยใช้วิธีการของคิว และการดูแลระบบของงานที่ถูกเสนอไปแล้วด้วยรูปแบบที่ condor มีอยู่แล้ว



รูปที่ 3-5 ตำแหน่งของ Scheduler ในระบบกริด

3.1.5. Data Management (การจัดการข้อมูลในระบบกริด)

เมื่อมีการติดต่อกับภายในกริด สิ่งหนึ่งที่ต้องให้ความสำคัญคือ การเคลื่อนย้ายข้อมูลหรือไฟล์ระหว่างแต่ละจุดในระบบ ทั้งนี้ต้องให้การเคลื่อนย้ายที่เกิดขึ้นมีความเชื่อถือได้และความปลอดภัย ซึ่งใน globus จะมีบริการที่ทำหน้าที่นี้ คือ Grid access to Secondary Storage (GASS) อาทิเช่น GridFTP สร้างขึ้นบนมาตรฐานของ FTP โพรโทคอล และเพิ่มฟังก์ชันบางอย่างเพื่อให้สามารถทำงานร่วมกับ GSI ที่มีอยู่แล้วได้

- **GridFTP**

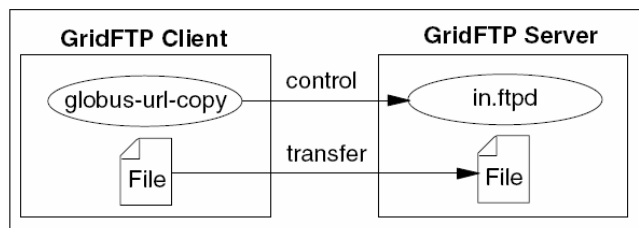
GridFTP เพิ่มความสามารถจาก โพรโทคอล FTP เดิมคือ

1. สนับสนุนการใช้งานคู่กับ GSI ของ globus toolkit และ Keberos และสามารถใช้ในการพิสูจน์ตัวตนได้ทั้งสองแบบ ผู้ใช้ระบบกริดสามารถเลือกระดับของความถูกต้องของข้อมูล และการรักษาความลับให้แตกต่างกันได้
2. อนุญาตให้มีการส่งไฟล์ระหว่างเครื่องเซิร์ฟเวอร์ได้ (Third-party data transfer)
3. สามารถส่งข้อมูลแบบขนานได้โดยใช้สายสตรีมของ TCP ได้หลายสายซึ่งจะสามารถรวมแบนด์วิดท์ได้
4. ส่งไฟล์บางส่วนได้ คือแบ่งแยกไฟล์และทำการส่ง
5. มีการส่งข้อมูลที่เชื่อถือได้ เมื่อเกิดความผิดพลาดสามารถกู้คืนข้อมูลได้
6. สามารถกำหนดขนาดของบัฟเฟอร์ของโปรโตคอล TCP ได้

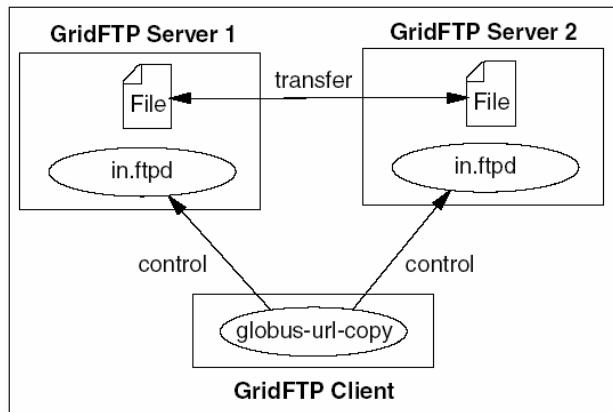
ขั้นตอนการทำงาน ใน Globus toolkit มีส่วนของ GridFTP ของเครื่องเซิร์ฟเวอร์และเครื่องไคลเอนท์ ซึ่งสร้างขึ้นโดย in.ftpd daemon และใช้ร่วมกับคำสั่ง globus-url-copy

เครื่องเซิร์ฟเวอร์และเครื่องไคลเอนท์ของ GridFTP จะสนับสนุนการส่งข้อมูล 2 ลักษณะคือ

- Standard



- Third Party

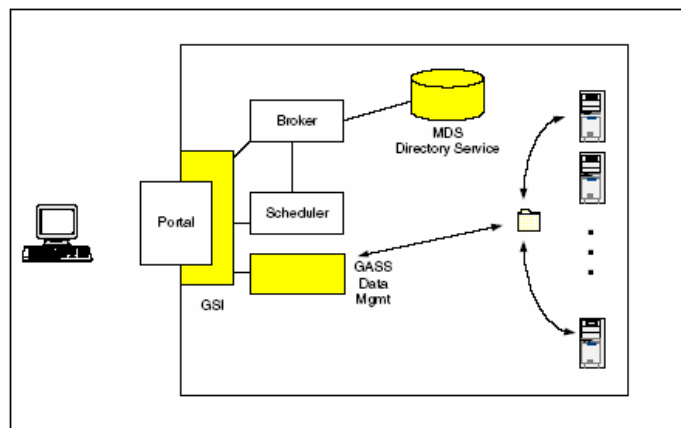


- **Globus Access to Secondary Storage (GASS)**

GASS ใช้ในการส่งข้อมูลระหว่าง GRAM ของเครื่องไคลเอนท์ และ GRAM ของเครื่องเซิร์ฟเวอร์ GASS ยังทำหน้าที่ในการเปิด , ปิด หรือเลือกข้อมูลจากกลุ่มของข้อมูลในระบบกริด GASS ช่วยกำจัดความต้องการที่จะเข้าไปยังแหล่งข้อมูลนั้น และส่งไฟล์ ตลอดจนการติดตั้งไฟล์ที่ทำหน้าที่ในการกระจายระบบ

- **Replica Management**

สามารถเก็บสำเนาของแหล่งเก็บข้อมูลไว้ที่แหล่งเก็บข้อมูลของเราเองเพื่อให้เกิดความรวดเร็วในการเข้าถึงข้อมูลที่เราต้องการ



รูปที่ 3-6 ตำแหน่งของ GASS ในระบบกริด และการใช้ GridFTP

3.1.6. Job and Resource Management (การจัดการงานและทรัพยากรในระบบกริด)

บริการที่ทำหน้าที่เป็นส่วนหลักของการทำงานในระบบกริด คือ Resource Allocation Manager (GRAM) ทำหน้าที่ในการดูแลทรัพยากรและจัดการกับทรัพยากรโดยตรง โดยการตรวจสอบสถานะของทรัพยากร และ รับผิดชอบเมื่องานนั้นสำเร็จ

เมื่อเครื่องไคลเอนต์ต้องการทำงาน จะส่งคำขอร้องไปยังเครื่องที่อยู่ระยะไกล และจะมีผู้ดูแลคือ gatekeeper ซึ่ง gatekeeper จะสร้างผู้ดูแลงานเหล่านั้นและเมื่องานนั้นเสร็จสิ้นลง ผู้ดูแลงานนั้นจะส่งรายงานสถานะไปยังเครื่องไคลเอนต์ และทำการสิ้นสุดงานนั้น

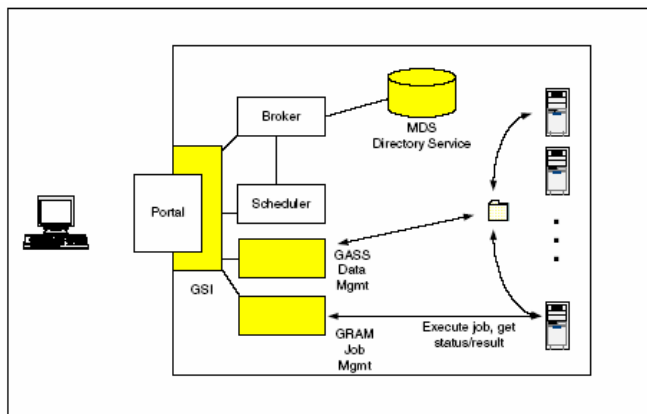
GRAM จะประกอบไปด้วยระบบย่อยๆ คือ

- คำสั่ง *globusrun* ซึ่งใช้ติดต่อกับแอปพลิเคชัน เป็นคำสั่งที่ใช้ในการเสนองาน โดยการติดต่อกับทรัพยากร คำสั่งนั้นจะถูกส่งไปในรูปแบบของ RSL สตริง ที่ระบุพารามิเตอร์และคุณสมบัติอื่นๆที่ต้องใช้ในการทำงานให้สำเร็จ
- *Resource Specification Language (RSL)* คือภาษาที่เครื่องไคลเอนต์ใช้ในการระบุงานที่ต้องการทำ ในการร้องขอและเสนองานจะใช้ RSL ในการอธิบายความต้องการของเครื่องไคลเอนต์ เช่นไฟล์ที่ต้องใช้ในการทำงาน,พารามิเตอร์ และข้อมูลอื่นๆที่เกี่ยวข้องกับการทำ redirection ไปยัง stdin , stdout, stderr โดยปกติแล้วจะมีมาตรฐานในการระบุทุกๆข้อมูลที่ต้องการในการทำงาน ซึ่งไม่ขึ้นกับองค์ประกอบอื่นๆ ซึ่งจะเป็นหน้าที่ของผู้จัดการงานในการแบ่งแยกข้อมูลที่ได้รับ และเริ่มต้นทำงานในแนวทางที่เหมาะสม
- *Gatekeeper Daemon* เป็นตัวจัดการการติดต่อที่มีกลไกการรักษาความปลอดภัยระหว่างเครื่องเซิร์ฟเวอร์และเครื่องไคลเอนต์ gatekeeper จะใช้ GSI ในการพิสูจน์ตัวตนก่อนที่จะสามารถทำงานได้ หลังจากการพิสูจน์ตัวตนสำเร็จ gatekeeper จะเริ่มต้นทำงานจริง และจะให้อำนาจกับเครื่องไคลเอนต์ในการติดต่อกับระบบกริด
- *ผู้จัดการงาน (job manager)* ถูกสร้างโดย gatekeeper daemon ซึ่งเป็นส่วนหนึ่งของกระบวนการร้องขอให้ทำงาน ผู้จัดการงานจะมีหน้าที่ติดต่อก็คือส่วนของการควบคุมการจองทรัพยากร ซึ่งอาจจะอยู่ในบริการที่เรียกว่า Job Scheduler หน้าที่การทำงานของผู้จัดการงานคือ
 - a) ทำการแยกแยะ RSL String ที่ส่งมาจากเครื่องไคลเอนต์
 - b) ทำการจองทรัพยากรให้เหมาะสมกับงานที่ร้องขอมา
 - c) ส่งข้อมูลกลับไปหาเครื่องไคลเอนต์ (ในกรณีที่เป็น)
 - d) ได้รับสถานะจากเครื่องไคลเอนต์ คือ สถานะร้องขอ หรือ สถานะที่ต้องการยกเลิก
 - e) ส่งผลลัพธ์ไปยังเครื่องไคลเอนต์โดยใช้ GASS

- ***Dynamically-Updated Request Online Coallocator (DUROC)*** เป็นแอปพลิเคชันที่อนุญาตให้ผู้ใช้ระบบกริดสามารถรับงานหลายงานได้ด้วยคำสั่งเดียว DUROC จะใช้ coallocator เพื่อใช้ในการคำนวณงานและจัดการงานผ่านผู้จัดการทรัพยากรหลายๆ แห่งใน globus สามารถเรียกใช้ DUROC ด้วยคำสั่ง globus-duroc RSL script จะมีส่วนประกอบที่มีความหมายของ DUROC ด้วย ซึ่ง GRAM ที่เครื่องไคลเอนต์จะทำการแยกแยะความหมายของ DUROC และทรัพยากรนั้นจะถูกจองด้วยการทำงานของผู้จัดการงานหลายตัว

การพิจารณาแอปพลิเคชันเมื่อเป็นมุมมองของการจัดการทรัพยากร

- การเลือกทรัพยากรที่เหมาะสมกับแอปพลิเคชันbroker ทำหน้าที่เลือกทรัพยากรที่เหมาะสมและเป็นไปตามความต้องการของแอปพลิเคชัน และผู้พัฒนาแอปพลิเคชันจะต้องสามารถกำจัด ข้อจำกัดบางอย่างของแอปพลิเคชันได้ เพื่อให้เป็นการสร้างโอกาสที่เพิ่มขึ้นที่จะจองทรัพยากรที่ว่างอยู่ได้มากขึ้น
- งานย่อยๆจำนวนมาก หากแอปพลิเคชันมีงานหลายงาน ผู้ออกแบบจะต้องเข้าใจความสัมพันธ์และความเกี่ยวเนื่องภายในงานย่อยๆเหล่านั้นจึงต้องหาตรรกะเพื่อมารองรับข้อ จำกัดเหล่านี้ อันได้แก่
 - a) Inter-process Communication
 - b) การใช้ข้อมูลร่วมกัน
 - c) การทำงานขนานกันของงาน
- การเข้าถึงผลลัพธ์ ถ้างานเหล่านั้นได้ส่งค่าของสถานะ หรือผลลัพธ์จำนวนหนึ่งออกมา แอปพลิเคชันสามารถจะรับข้อมูลเหล่านั้นได้จาก stdout และ stderr
- ผู้จัดการงาน GRAM มีกลไกในการค้นหาสถานะของงาน แอปพลิเคชันต้องใช้ความสามารถของผู้จัดการงาน เพื่อบอกให้ผู้ใช้ระบบทราบได้ว่าควรจะทำอย่างไรกับทรัพยากร เช่น เมื่อ งานๆหนึ่งผิดพลาด งานอื่นๆที่มีความเกี่ยวเนื่องกับงานชิ้นนี้ย่อมต้องยกเลิกด้วยและทำให้ต้องทำการปล่อย (free) ทรัพยากรนี้นั่นเอง



รูปที่ 3-7 ตำแหน่งของ GRAM ในระบบกริด

3.2. GLOBUS Toolkit (GT3)

3.2.1. GT3 คืออะไร

GT3 (Globus Toolkit 3) นั้นเป็นเครื่องมือที่พัฒนาโดยภาษา Java ที่นำคุณสมบัติของ OGSI มาใช้ในการพัฒนา กล่าวคือ GT3 เปรียบเสมือนเฟรมเวิร์คที่ใช้ในการสร้างกริดเซอร์วิส

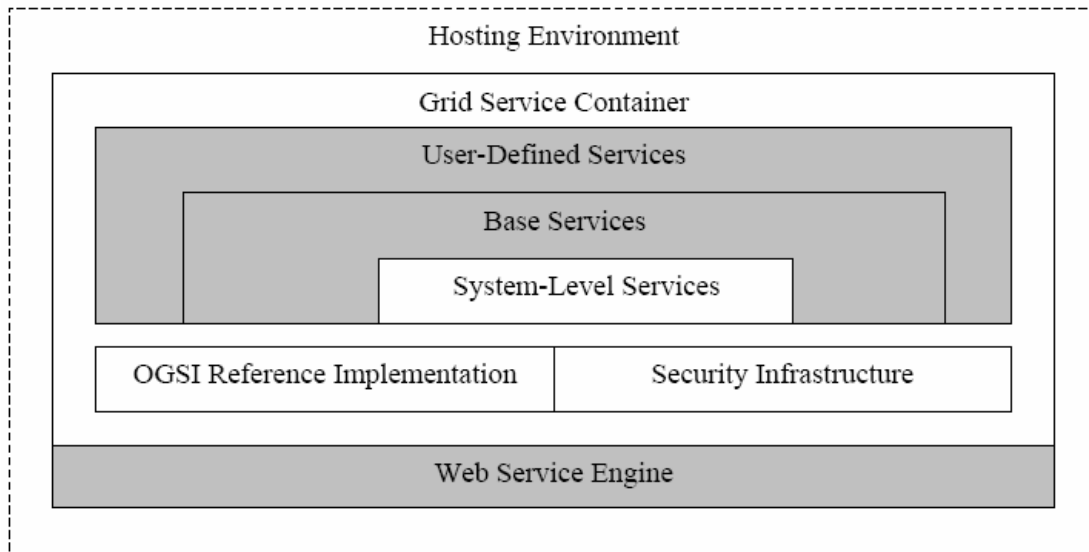
3.2.2. เซอร์วิสพื้นฐานที่จัดเตรียมโดย GT3

GT3 นั้นได้จัดเตรียมเซอร์วิสพื้นฐานไว้มากมาย ซึ่งแบ่งออกเป็นส่วนๆตามหน้าที่ ได้ดังต่อไปนี้

- Core (แกน)
- Security (ด้านความปลอดภัย) อันได้แก่ GSI และ CAS
- Data Management (การจัดการข้อมูล) อันได้แก่ GridFTP, RLS, RFT, XIO
- Resource Management (การจัดการทรัพยากร) อันได้แก่ GRAM
- Information service (การให้บริการด้านข้อมูล) อันได้แก่ MDS

3.2.2.1. แกน (Core)

ภาพด้านล่างนี้ เป็นภาพโครงสร้างของ GT3 ซึ่ง องค์ประกอบของ Core จะถูกแทนในช่องสีขาว



รูปที่ 3-8 โครงสร้างของ GT3

3.2.2.1.1. Hosting Environment

คือ กลุ่มโฮสต์ที่ใช้ในการสร้างระบบกริด ได้จำแนกไว้ 4 ประเภทที่แตกต่างกัน คือ

1. Embedded โดยจะมีประโยชน์กับไคลเอนต์หรือเซิร์ฟเวอร์ขนาดเล็กที่ให้ใช้งานกริดเซอร์วิส
2. Standalone มักใช้กับโปรแกรมขนาดเล็ก
3. Servlet ใช้พื้นฐานของ Java Servlet Engine มาเป็นมาตรฐานของ Container
4. EJB (Enterprise Java Bean) ใช้ EJB application server มาช่วยในการทำระบบกริด

ซึ่งทุกประเภทของ Hosting environment ที่กล่าวไปข้างต้น สามารถรองรับการทำกริดเซอร์วิสได้ โดยแบบที่ 3 นั้นจะเป็นการสร้างกริดเซอร์วิสแบบชั่วคราวจริงๆ แต่แบบที่ 4 จะชั่วคราวที่การทำงานในส่วนของ EJB

3.2.2.1.2. Grid Service Container

เป็นส่วนที่จะใช้เก็บเซอร์วิสต่างๆ ได้แบ่งการทำงานออกเป็น 3 ประเภท ดังนี้

1. คิดเกี่ยวกับเซอร์วิสขนาดเล็กๆ โดยใช้การค้นหา 2 แบบ คือ pull และ push
2. ควบคุมซอฟต์แวร์ของเซอร์วิสอินสแตนซ์ที่ทำงานเป็นสเตทอย่างเต็มรูปแบบซึ่งอ้างได้อย่างทั่วถึง
3. ขนส่งส่วนที่เป็นอิสระกับโครงสร้างการทำงานที่เน้นความปลอดภัยตาม Grid Service Infrastructure โดยใช้ delegation, message signing และ encryption มาใช้ในการอนุญาต

3.2.2.1.3. OGIS Reference Implementation

เป็นส่วนที่มาจาก OGSI ซึ่งเป็นนำคุณสมบัติทั้งหมดตามที่กำหนดไว้ใน OGSI มาช่วยในการสร้างระบบ เช่น GridService, Factory, Notification (Source/Sink/Subscription), HandleResolver, ServiceGroup (Entry/Registration)

3.2.2.1.4. Security Infrastructure

เป็นส่วนที่ใช้เกี่ยวกับความปลอดภัยในระดับการส่งข้อความ โดยจะมีองค์ประกอบของความปลอดภัยระดับข้อความ, การรับรองสิทธิ และการอนุญาตขึ้นกับกริดแม่พิมพ์ ซึ่งสิ่งที่น่าสนใจในข้อนี้ก็มี WS-Security, XML-Signature และ XML-Encryption

3.2.2.1.5. System Level Services

เป็นกริดเซอร์วิสที่ทำตาม OGSI ซึ่งสามารถใช้งานได้โดยโดยกริดเซอร์วิสตัวอื่นๆทั้งหมด ซึ่ง GT3 ในตอนนี้ ได้แบ่งเซอร์วิสส่วนนี้ออกเป็น 3 ระดับ

- Ping service: ใช้ในการ “ping” ไปยัง hosting environment เพื่อเช็คเข้าถึงได้หรือไม่
- Logging Management Service: ยอมให้แก้ไขการกรอง log และใช้สร้าง log ที่คงอยู่เพื่อช่วยในการจัดการเวลาทำงาน
- Management Service: เป็นส่วนจัดเตรียมการมอนิเตอร์สถานะและการไหลของกริดเซอร์วิสคอนเทนเนอร์ และเพื่อใช้ปิดคอนเทนเนอร์ และยังเป็นส่วนที่ยอมให้สั่งเปิดและปิดเซอร์วิสอินสแตนซ์

3.2.2.2. ด้านความปลอดภัย (GSI และ CAS)

3.2.2.2.1. GSI (Grid Security Infrastructure)

Grid Security Infrastructure (GSI) เป็นการเข้ารหัสลับสาธารณะ เพื่อใช้ประโยชน์ในด้านความปลอดภัยต่างๆ โดยจุดประสงค์เบื้องต้นที่ทำให้เกิด GSI ขึ้นนั้น คือ

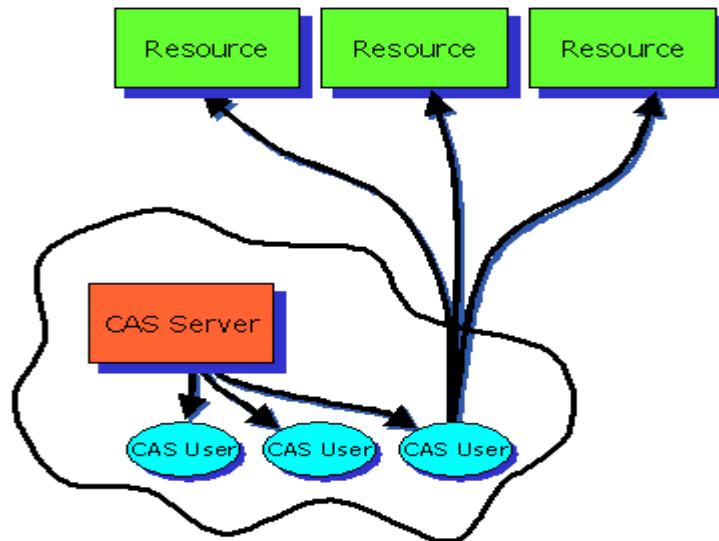
- ความต้องการการสื่อสารที่ปลอดภัย รับรองได้ บางทีต้องเป็นความลับระหว่างการสื่อสารที่ใช้ระบบกริด
- ความต้องการที่จะรองรับการใช้งานข้ามเขตของแต่ละองค์กร ภายใต้ระบบความปลอดภัยที่ควบคุมจากศูนย์กลาง
- ความต้องการที่จะรองรับ “single sign-on” สำหรับผู้ใช้งานกริด โดยที่การสื่อสารนั้น จะครอบคลุมไปถึงการใช้งานหลายๆริชอร์สหรือจากหลายๆไซต์ได้

ซึ่งรายละเอียดต่างๆของ GSI มีความซับซ้อนมากจึงขอลำถึงในบทถัดไป (บทที่ 4)

3.2.2.2.2. CAS

CAS สร้างขึ้นโดย Globus Toolkit ใช้ใน GSI ซึ่ง CAS เป็นส่วนที่ยอมให้ส่วนที่จัดเตรียม

ริชอร์ส ยอมให้มีการสื่อสารโดยควบคุมด้วยนโยบายสำหรับการเข้าถึงมัน ซึ่งจะใช้หลักของคิเล็กชั่น (delegation) ในการควบคุมการจัดการในส่วนนี้



รูปที่ 3-9 แสดงการทำงานของ CAS

การทำงานของ CAS

1. CAS Server จะเริ่มสร้างช่องทางการสื่อสาร โดยจะได้มาซึ่ง GSI ที่เตรียมไว้สำหรับให้ CAS server กำหนดการสื่อสาร
2. ตัวจัดเตรียมริชอร์สจะได้สิทธิสำหรับการสื่อสารนั้นๆ โดยแต่ละตัวจัดเตรียมนั้นจะเปรียบเสมือนผู้ถือครองการสื่อสารและมีนโยบายการสื่อสารเป็นของตนเอง เมื่อสร้างการเชื่อมต่อที่เชื่อถือได้แล้ว ตัวจัดเตรียมริชอร์สก็จะยอมให้มีหลักฐานสำหรับการสื่อสารนั้น โดยใช้วิธีการภายในของมัน (เช่น ใช้ กริดแม่ไฟฟ์และดิสก์ไควด้า, การยินยอมจากระบบไฟล์ เป็นต้น)
3. ตัวแทนการสื่อสารนั้นจะใช้ CAS เพื่อจัดการความเชื่อถือได้ของการสื่อสารนั้น และยอมให้ได้รับสิทธิการเข้าถึงริชอร์สนั้น โดย CAS server จะมียุทธศาสตร์ของมันที่ใช้ในการจัดการ เช่น สมาชิกของการสื่อสารต้องการสิทธิเพื่อเพิ่มสมาชิกของการสื่อสารนั้น หรือ ขอสิทธิสำหรับเข้าถึงริชอร์สทั้งหมดของการสื่อสารนั้น
4. เมื่อผู้ใช้งานต้องการที่จะเข้าถึงริชอร์สได้ร้องขอไปยัง CAS server แล้ว ถ้าฐานข้อมูลของ CAS server ยอมให้ผู้ใช้นั้นใช้งาน CAS จะแจก GSI restricted proxy เพื่อให้ผู้ใช้งานสามารถทำได้ตามที่ร้องขอไว้
5. เมื่อผู้ใช้ได้รับการเชื่อมต่อกับริชอร์สของ CAS ด้วยเครื่องมือปกติของ Globus แล้ว (เช่น GridFTP) ริชอร์สที่ผ่านการยอมรับก็จะส่งไปยังผู้ที่ได้รับการยินยอมให้เข้าถึง และได้รับสิทธิ

การเข้าถึงตามที่จำกัดไว้ตามนโยบายของ CAS โดยจะมีข้อจำกัดตามแต่สิทธิของผู้ใช้แต่ละคน ซึ่ง CAS ยินยอมให้ตัวจัดเตรียมรีพอร์ตสื่อสารด้วย

3.2.2.3. Data Management (การจัดการข้อมูล)

3.2.2.3.1. GridFTP

GridFTP เป็นโปรโตคอลที่ใช้ในการส่งข้อมูลไปบน wide-area networks ที่มีประสิทธิภาพ, ปลอดภัย และเชื่อถือได้ ซึ่ง GridFTP นั้นมีพื้นฐานมาจาก FTP ซึ่งเป็นโปรโตคอลที่นิยมใช้ในการส่งข้อมูลในระบบอินเทอร์เน็ต ซึ่งผู้พัฒนาได้เพิ่มและขยายส่วนต่างๆที่จำเป็นสำหรับการใช้ในระบบกริด

ซึ่งคุณสมบัติที่ GridFTP จัดเตรียมไว้ในระบบกริด มีดังต่อไปนี้

- ทำงานร่วมกับ GSI security เพื่อควบคุมช่องทางข้อมูลให้ปลอดภัย
- ส่งข้อมูลแบบหลายๆทางแบบขนานไปพร้อมๆกัน
- เลือกส่งไฟล์บางส่วน
- ส่งไปยังกลุ่มที่ 3 (ส่งตรงจากเซิร์ฟเวอร์ไปยังเซิร์ฟเวอร์)
- รับรองช่องทางข้อมูล
- ใช้ช่องทางข้อมูลใหม่
- ทำไปป์ไลน์ของคำสั่งได้

3.2.2.3.2. RLS

Replica Location Service (RLS) เป็นส่วนที่ดูแลและจัดเตรียมการเข้าสู่การแม็พจากข้อมูลที่เป็นลอจิกให้เป็นค่าข้อมูลที่จะใช้ที่เป้าหมาย ซึ่งส่วนนี้ได้พัฒนาโดยทีมที่ทำเรื่อง DataGrid ซึ่งจะช่วยลดเวลาในการเข้าถึงข้อมูล เพิ่มการวางตำแหน่งที่ตั้งของข้อมูล และเพิ่มความแข็งแรง, ประสิทธิภาพของแอปพลิเคชันแบบกระจาย

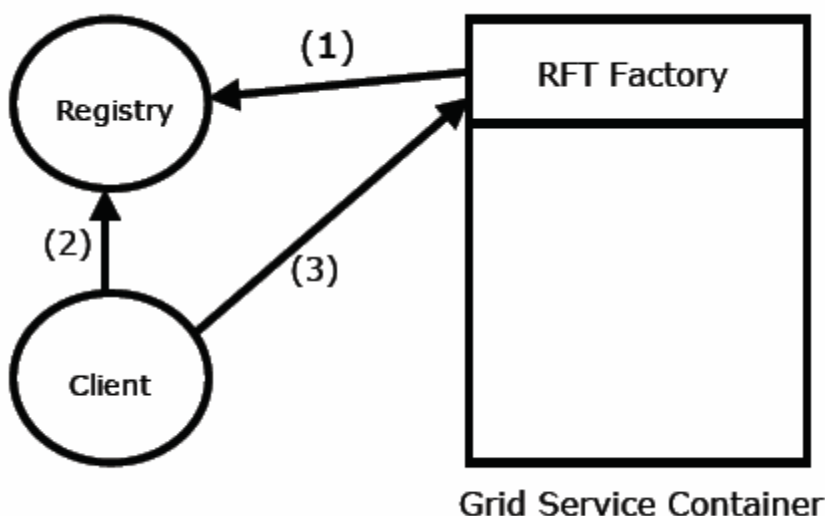
RLS นั้นจะมีหน้าที่ในด้านต่อไปนี้

- Consistent local state maintained in Local Replica Catalogs (LRCs): จะใช้ดูแลการแม็พระหว่าง arbitrary logical file names (LFNs) และ physical file names (PFNs) โดยเกี่ยวข้องกับระบบของอุปกรณ์การเก็บข้อมูล
- Collective state with relaxed consistency maintained in Replica Location Indices (RLIs): แต่ละ RLI จะประกอบไปด้วยกลุ่มของการแม็พจาก LFNs เป็น LRCs
- Soft state maintenance of RLI state: LRC จะส่งข้อมูลที่เกี่ยวข้องกับสเตทของ RLIs โดยใช้ซอฟต์แวร์โปรโตคอล ซึ่งค่าข้อมูลสเตทนั้นจะต้องรีเฟรชอย่างสม่ำเสมอ

- Compression of state update: ใช้รวมเนื้อหาทั้งหมดของ Local Replica Catalog ก่อนที่จะส่งไปยังการอัปเดตแบบซอฟต์สเตทที่ Replica Location Index Node
- Membership and partitioning information maintenance: RLS จะใช้ดูแลข้อมูลที่เป็นแบบ static ที่เกี่ยวกับ LRCs และ RLIs ในระบบแบบกระจาย ซึ่งจะใช้กลไกของ OGSA สำหรับการลงทะเบียนเซอร์วิสและใช้จัดการช่วงเวลาชีวิตของเซอร์วิสนั้น

3.2.2.3.3. RFT

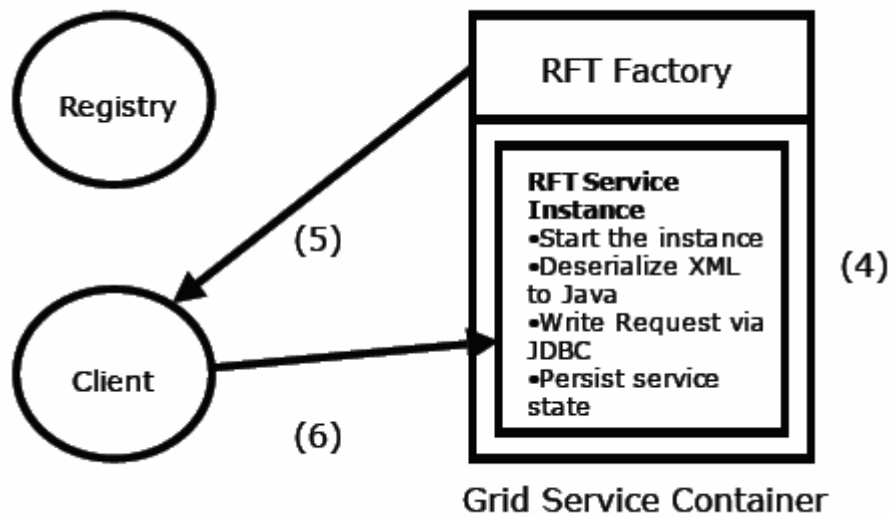
Reliable File Transfer (RFT) เป็นเซอร์วิสพื้นฐานของ OGSA ที่ใช้สำหรับควบคุมดูแลการถ่ายโอนข้อมูลระหว่างผู้ใช้งานกลุ่มที่ 3 ที่ใช้ GridFTP ซึ่งมีหลักการทำงานดังภาพ



รูปที่ 3-10 แสดงขั้นตอนการทำงานของ RFT

การทำงานเริ่มเมื่อกริดเซอร์วิสคอนเทนเนอร์เริ่มทำงาน โดยบรรจุ RFT factory ไว้

1. RFT Factory นั้นจะเริ่มลงทะเบียนตัวของมันให้เป็นคำริจิสตรีที่เป็นที่รู้จักกัน
2. ไคลเอนท์ค้นเจอ RFT Factory นั้น โดยการคิวรี serviceData ของริจิสตรีนั้น
3. ไคลเอนท์เรียกคำสั่ง createService บนแพลตฟอร์มนั้นและส่งไปยังที่ร้องขอ



รูปที่ 3-11 แสดงขั้นตอนการทำงานของ RFT (ต่อ)

4. เซอร์วิสอินสแตนซ์นั้นเริ่มทำงาน
5. RFT factory จะส่งค่า locator กลับไปยังไคลเอนท์
6. ไคลเอนท์ จะเรียก Start() และส่งค่าไปยังส่วน notifications หรืออื่นๆ

3.2.2.3.4. XIO

Globus XIO เป็นส่วนที่ใช้สำหรับขยายอินพุต/เอาต์พุต ไบเบรารีสำหรับ Globus Toolkit ที่ใช้จัดเตรียม API พื้นฐาน (เช่น open/close/read/write) เพื่อนำไปใช้ใน IO ตามต้องการ ซึ่ง Globus XIO นั้นมีจุดหมายสำคัญอยู่ 2 ประการ คือ

- เตรียม API สำหรับผู้ใช้แบบเดียวให้กับโปรโตคอลที่เกี่ยวข้องกับ IO ของกริด ส่วนนี้จะใช้การส่งข้อความสำหรับใช้งานในโปรโตคอลที่แตกต่างกัน ซึ่งผู้เขียนโปรแกรมจำเป็นต้องเขียนให้อยู่ในรูปแบบที่กำหนดไว้ใน API นั้นๆ ซึ่งโปรโตคอลที่สร้างให้ Globus XIO นั้นจะเรียกว่า “drivers” ซึ่งเมื่อใช้งานจะเป็นส่วนที่ซ่อนจากผู้พัฒนา โดยจำเป็นต้องเขียน API นี้ในครั้งแรกเมื่อจะใช้แอปพลิเคชันนั้นๆในครั้งต่อไป จะสามารถทำได้โดยไม่ต้องเปลี่ยนซอร์สโค้ดเลย
- ลดเวลาในการสร้างหรือทดลองโปรโตคอลใหม่ๆ โดย Globus XIO นั้นจะจัดเตรียม “driver” ดังที่กล่าวไปแล้ว ทำให้ไม่จำเป็นต้องยุ่งมากนักในการใช้โปรโตคอลใหม่ๆ และยังเป็นส่วนที่ทำให้เกิดการนำซอร์สโค้ดเก่ามาใช้งานใหม่ด้วย และยังสามารถนำไปรวมกับ driver อื่นๆได้

Globus XIO นั้นมีฟีเจอร์ที่เกี่ยวข้องกับระบบการทำงาน โดยได้เตรียม API ที่ใช้งานได้ให้กับผู้ใช้งาน ทำให้ผู้ใช้งานไม่ต้องเขียนฟังก์ชันยากๆด้วยตัวเองอย่างเดียว โดยฟีเจอร์ของ XIO นั้น มีดังนี้

- Simple API ใช้สำหรับงานพื้นฐาน เช่น open, close, read, write
- Extensibility with drivers เนื่องจาก Globus XIO นั้นออกแบบให้ขยายได้ง่าย ทำให้สามารถเพิ่มฟีเจอร์ใหม่ๆ เข้าไปได้
- Efficiency เนื่องจาก Globus XIO ได้ถูกออกแบบให้มีประสิทธิภาพ มีไลบรารีที่สามารถใช้งานได้หลากหลาย
- Timeouts ทำให้ผู้ใช้งานไม่ต้องกำหนด timeout เอง เพราะ globus xio จะมีค่า timeout โดยปกติของมันอยู่แล้ว และสามารถปิดตัวเองเมื่อทราบว่าคำสั่งนั้นไม่ต้องทำงานแล้วด้วย
- Data Descriptions มีบัพเฟอร์ที่ให้ผู้ใช้งานสามารถเขียนหรืออ่านได้ โดย Globus XIO ได้ยอมให้ผู้ใช้งานเข้าไปจัดการส่วนนี้ได้ แม้ว่า IO นั้นจะทำงานสมบูรณ์ไปแล้ว

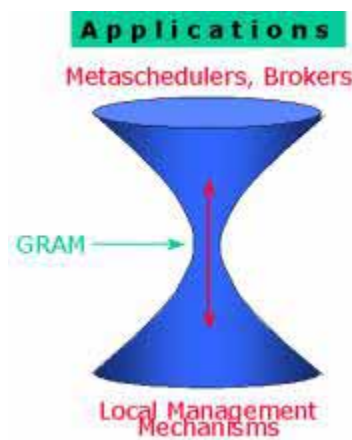
3.2.2.4. Resource Management (การจัดการทรัพยากร)

3.2.2.4.1 GRAM

Globus Toolkit ได้ใช้ GRAM (Grid Resource Allocation and Management) ในการจัดการระบบในระยะไกลโดยเตรียมรูปแบบสำหรับการร้องขอและใช้ระบบจากระยะไกลสำหรับการทำงาน โดยส่วนที่ใช้มากที่สุดก็คือ ใช้สำหรับจ่ายและควบคุมงานจากระยะไกล

GRAM ได้ออกแบบโดยใช้โปรโตคอลพื้นฐานและ API สำหรับการร้องขอและใช้รีซอร์สจากระบบในระยะไกล โดยอาศัยรูปแบบที่อ้างอิงตารางงาน (Job Schedule) โดยอาศัย GSI มาช่วยเสริมในด้านความปลอดภัย

GRAM จะใช้หลากหลายวิธีในการควบคุมงาน คือใช้ ตารางงาน, ระบบคิวรี, ระบบสำรอง และรูปแบบการควบคุม ซึ่งทำให้ผู้พัฒนาสามารถเรียนรู้มันและนำไปช่วยในการพัฒนาระบบได้ ซึ่งถ้าเปรียบแล้ว GRAM จะทำงานเหมือนกับคอกของนาฬิกาทรายที่ใช้แอปพลิเคชันและเซอร์วิสระดับสูง (เช่น resource broker) มาช่วยในการให้สิทธิการเข้าถึงนั่นเอง



รูปที่ 3-12 ลักษณะการทำงานของ GRAM

3.2.2.5. Information Service (การให้บริการข้อมูล)

3.2.2.5.1. MDS3

Monitoring and Discovery Services (MDS3) เป็นส่วนที่จัดการเกี่ยวกับข้อมูลเซอวิซ สำหรับ Globus Toolkit 3 ซึ่งจะจัดการกับพวก serviceData และ Index Service โดยมีวิธีการคิวรีผ่าน XML รูปแบบหนึ่งที่เรียกว่า XPath

serviceData ที่ใช้ใน GT3 ก็คือ ข้อมูล XML ที่ถูกสร้างโดยทุกๆกริดเซอวิซที่แทนในสเตทภายใน ซึ่งแต่ละส่วนของ serviceData นั้นจะถูกเรียกว่า serviceData Element (SDE) ดังที่กล่าวไปแล้วในบท OGSI ซึ่งสเตทของโฮสต์นั้นๆจะถูกแสดงโดย SDE แบบเดี่ยว (ด้วย GRAM) หรือจากสถานะของงาน

วิธีการคิวรีแบบ XPath ที่ใช้ใน MDS นั้น สร้างขึ้นมาจาก XATAN XPath library ซึ่งเป็นส่วนหนึ่งของภาษาคิวรีแบบ W3C XML โดยที่เซอวิซใดๆที่ถูกสร้างขึ้นจาก GT3 OGSA แล้วจะต้องรองรับการทำงานของฟังก์ชันนี้ โดย Globus ได้กำหนดรูปแบบของการคิวรีอินพุต โดยใช้ค่า SDE และ XPath โดยที่เอาท์พุทที่ได้เป็นผลลัพธ์จากการคำนวณของ Xpath คิวรีนั้นกับกลุ่มของ SDE ที่พิจารณา